



RV College of
Engineering®

AI For Engineers

CI124TA

UNIT 4

Dr. Sindhu D V,

Assistant Professor,

Department of Computer Science and Engineering, RVCE

All material presented in this PowerPoint is credited to the respective authors of standard textbooks, reference presentations shared by senior faculty at **RVCE**, and authentic internet sources.

Note to Students:

This presentation is meant **only for classroom reference and conceptual understanding**.

For exam and detailed preparation, please refer to the **textbook extracts and materials shared in Google Classroom**, as the content here is a **summary version** and not exhaustive.

Go, change the world®



Why did scientists look at the brain?

It's the 1940s. Computers exist, but they can only do what you *exactly* tell them to do — follow rigid rules, one step at a time.

Scientists had a problem: some tasks are impossible to write rules for. How do you write a rule for "recognise a dog in a photo"? Dogs come in every shape, size, and colour. There are too many exceptions.

Then they looked at the human brain. The brain doesn't follow written rules. It *learns from examples*. Show a child 20 dogs, and they can recognise any dog they've never seen before.

The big question: **Can we build a machine that learns the way the brain does?** The answer was
→ yes — and that's where Artificial Neural Networks came from.



What exactly is a biological neuron?

Your brain has about **86 billion neurons**. Each one is a tiny decision-making cell. Here is what a single neuron does — think of it like a security guard at a gate:

Step 1 — Receive

The guard receives messages from many people trying to enter. These come through branches called *dendrites*.

Step 2 — Weigh

Not all messages are equal. A message from the police chief matters more than one from a stranger. This is *synapse strength*.

Step 3 — Decide

The guard adds up all the messages. If the total urgency is high enough, they open the gate. This happens in the *soma* (cell body).

Step 4 — Signal

If the gate opens, the guard sends a signal down the road to the next gate. This road is the *axon*.



Key idea

The most important part — the synapse

Between every two neurons is a tiny gap called a **synapse**. This gap can be:

Strong synapse

Signal passes through with full force. Like a wide-open highway — fast and clear.

Weak synapse

Signal barely gets through. Like a narrow lane — slow and faint.

When you **learn something** — say, riding a bicycle — your brain literally *physically changes* the strength of certain synapses. The more you practice, the stronger those connections get. That is memory. That is learning.

Famous saying in neuroscience: *"Neurons that fire together, wire together."* Practice strengthens connections. Forgetting weakens them.

This single insight — that learning = changing connection strength — is the entire foundation of Artificial Neural Networks.



A real-world example — learning to catch a ball

Imagine a child learning to catch a ball for the first time.

Attempt 1 — fails

Ball hits the child's face. Certain neurons fired, but with wrong timing. The brain notes: "that didn't work."

Attempt 10 — better

Hands are closer. Some neural paths got slightly stronger after repeated tries. Small improvement.

Attempt 50 — caught it!

The right synapse connections are now strong. The path for "eye sees ball → hands move to right position" is fast and reliable.

Attempt 1000 — automatic

The child doesn't even think about it. The neural path is so strong it runs on its own. This is muscle memory.

This is *exactly* how an ANN learns — it tries, fails, adjusts its "connection strengths" (weights), tries again, and gradually gets better. The algorithm that does this is called **backpropagation** — which we'll study later.

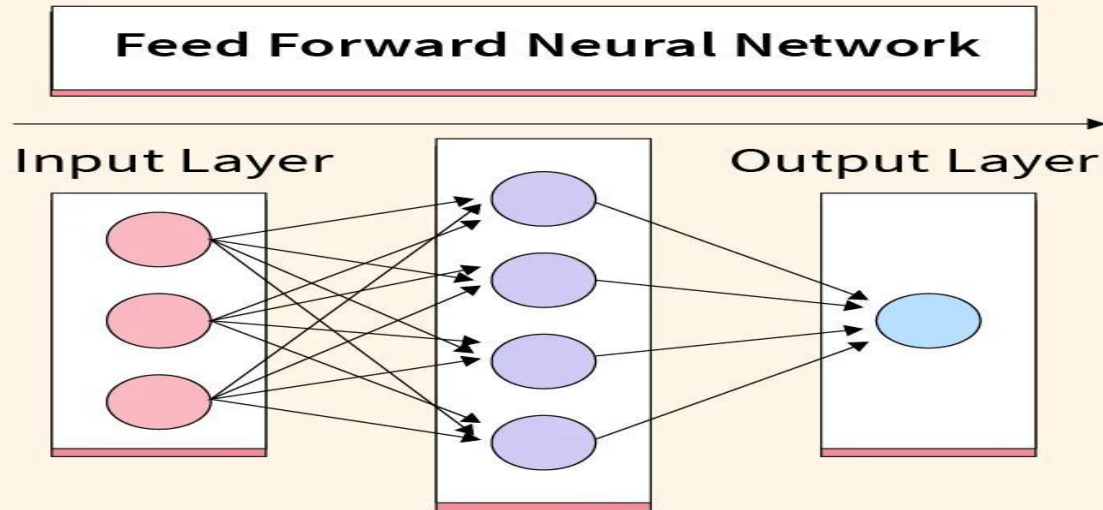


Brain → computer: what maps to what?

Now we make the connection explicit. Every part of a biological neuron has a mathematical twin in an artificial neuron:

Dendrite	→	Input (x)	A number: e.g. pixel brightness, temperature reading
Synapse strength	→	Weight (w)	A number the system learns and adjusts during training
Soma (cell body)	→	Weighted sum	Add up all (input × weight): $w_1x_1 + w_2x_2 + \dots$
Firing threshold	→	Activation function	If sum is big enough → output 1 (fire), else → 0
Axon → next neuron	→	Output → next layer	Passes result to the next artificial neuron

The biology is messy and chemical. The math is clean and simple. But the *logic* is identical.





A neural network has three main parts:

1. Input Layer

Receives information from the outside world.

Example:

- Study Hours
- Attendance
- Assignment Marks

2. Hidden Layer

Processes the information and finds patterns.

This layer is called "hidden" because we cannot directly see how it performs its calculations.

3. Output Layer

Produces the final result.

Example:

- Pass
- Fail



What is a Neural Network Representation?

A neural network represents knowledge using:

- Neurons
- Connections between neurons
- Weights (importance of connections)

Instead of storing rules, the network stores knowledge in the form of **weights** learned during training.

Example

To predict whether a student will pass:

Inputs:

- Attendance = 90%
- Internal Marks = 85
- Assignment Score = 80

The neural network processes these inputs and predicts:

Output = Pass



How Learning Happens

1. Input data is given to the network.
2. The network produces an output.
3. The output is compared with the correct answer.
4. Errors are calculated.
5. Weights are adjusted.
6. The process repeats until the network learns.

Just like a student improves after correcting mistakes, a neural network improves by reducing errors.



1. The Input Data Contains Many Attributes

Neural networks can handle a large number of input features.

Example:

- Student performance prediction using attendance, marks, assignments, projects, and participation.
- House price prediction using area, location, number of rooms, age of house, etc.



2. The Output Can Be Discrete or Continuous

Neural networks can predict:

Discrete Values (Classification)

Output belongs to a category.

Examples:

- Pass / Fail
- Spam / Not Spam
- Disease / No Disease

Continuous Values (Prediction)

Output is a numeric value.

Examples:

- House Price
- Temperature Forecast
- Sales Prediction



3. Training Data Contains Errors or Noise

Real-world data is often incomplete or contains mistakes.

Neural networks can still learn useful patterns even when data is noisy.

Example:

- Handwritten character recognition where different people write differently.
- Speech recognition with background noise.



4. The Relationship Between Inputs and Outputs is Complex

When it is difficult to write exact rules, neural networks are useful.

Example:

- Face Recognition
- Speech Recognition
- Image Classification

It is hard to create fixed rules for these problems, but neural networks can learn from examples.

5. Large Amounts of Training Data Are Available

Neural networks learn better when more examples are available.

Example:

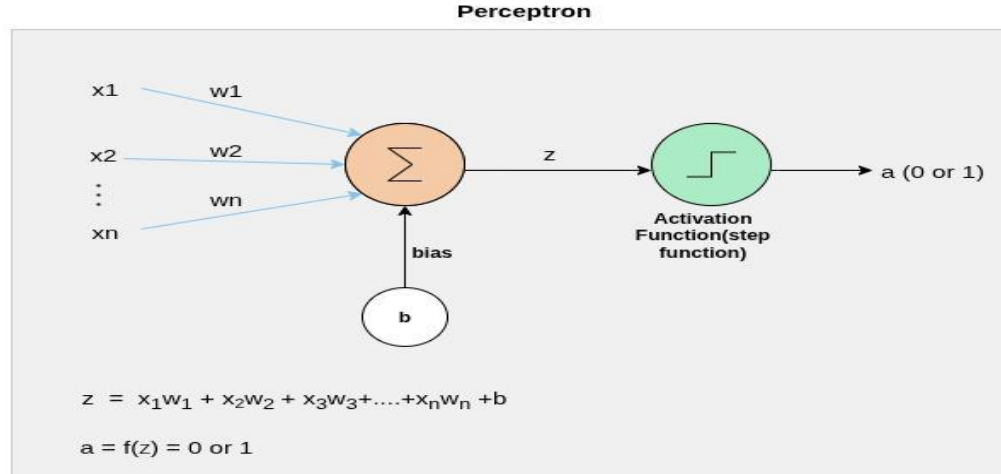
- Thousands of student records
- Millions of images for image recognition

More data generally improves learning performance.

- **What is a Perceptron?**

A **Perceptron** is the simplest type of artificial neural network and is considered the basic building block of neural networks.

It takes several inputs, performs a calculation, and produces a single output.





A perceptron consists of:

Inputs ($x_1, x_2, x_3, \dots$)

Features provided to the model.

Example:

- Attendance
- Marks
- Assignment Score

Weights ($w_1, w_2, w_3, \dots$)

Importance assigned to each input.

Example:

- Marks may be more important than attendance.

Bias (b)

A constant value that helps adjust the decision boundary.

Output (y)

Final prediction:

- 1 = Yes / Pass
- 0 = No / Fail



How Does a Perceptron Work?

Go, change the world®

Step 1: Receive Inputs

Step 2: Multiply by Weights

Each input is multiplied by its corresponding weight.

Step 3: Add Results

The perceptron calculates a weighted sum.

Step 4: Apply Activation Function

If the result is above a threshold:

Output = 1

Otherwise:

Output = 0

Step 5: Produce Decision

Example:

Pass = 1



$$o = f(\sum_{i=1}^n w_i x_i + b)$$

Where:

- x_i = inputs
- w_i = weights
- b = bias
- f = activation function
- o = output



Weight Update Rule

Definition

- The **Perceptron Training Rule** is a learning algorithm used to train a perceptron.
 - It adjusts the weights whenever the perceptron makes an incorrect prediction.
 - The goal is to minimize errors and improve classification accuracy.
-

How the Perceptron Learns

1. Provide input to the perceptron.
2. Calculate output using current weights.
3. Compare predicted output with actual output.
4. Update weights if the prediction is incorrect.
5. Repeat until the perceptron learns the correct classification.



Weight Update Rule

$$w_i \leftarrow w_i + \eta(t - o)x_i$$

Where:

- w_i = Current weight
- η = Learning rate
- t = Target (actual output)
- o = Predicted output
- x_i = Input value

Key Idea

- **Correct Prediction** → No weight change
- **Wrong Prediction** → Adjust weights
- The perceptron learns by correcting its mistakes.

What is Bias?

- Bias is an additional value added to a neuron.
- It acts like an intercept in a linear equation.
- Helps shift the decision boundary.
- Improves the learning capability of the network.

Output Calculation

$$Output = f(\sum wx + b)$$

where:

- w = Weight
- x = Input
- b = Bias

Analogy

Weight = Importance of input

Bias = Bonus marks / Starting value

A nice classroom sentence is:

"Weights decide how important an input is, while bias decides where the neuron starts its decision."



Activation Function

What is an Activation Function?

An **activation function** decides whether a neuron should be activated and what output it should produce.

After calculating:

$$Net = \sum wx + b$$

the activation function converts this value into the neuron's output.

Why Do We Need Activation Functions?

Suppose a network has many layers but uses only simple addition and multiplication.

Then the entire network behaves like a **single linear equation** and cannot learn complex patterns.

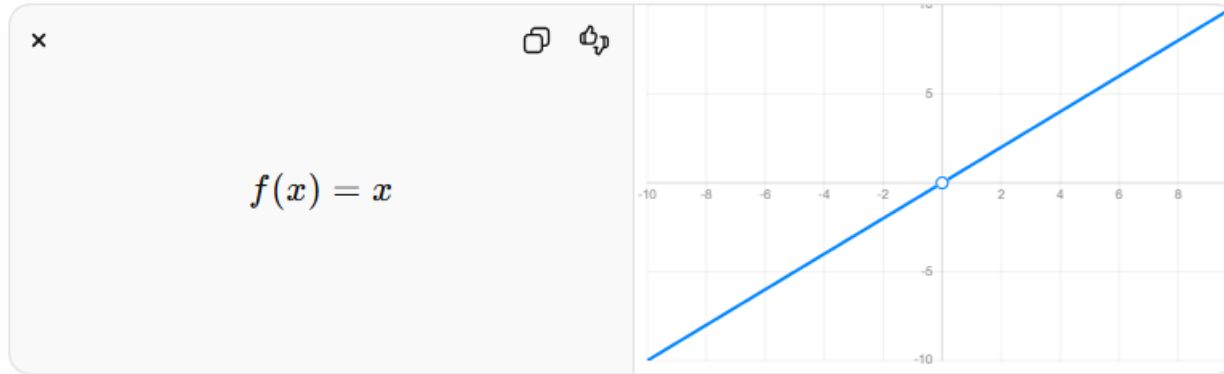
Activation functions introduce **non-linearity**, enabling neural networks to solve complex real-world problems.

Examples

- Face Recognition
- Speech Recognition
- Medical Diagnosis
- Self-Driving Cars



Linear Activation Function



Advantages

- Very simple.
- Easy to compute.

Limitations

- Cannot learn complex patterns.
- Multiple layers behave like a single layer.
- Rarely used in hidden layers.



● Non-Linear Activation Functions

These are the most important activation functions in neural networks.

2. Binary Step Function

Formula

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Output

Only:

- 0
- 1

Example

Input

Output

3

1

-2

0



Back Propagation algorithm

Go, change the world®

Step 1: Initialize Weights

- Assign small random values to all weights.

Step 2: Feed Forward

- Input is passed through the network.
- Hidden layer computes its output.
- Output layer produces the final prediction.

Step 3: Calculate Error

- Compare predicted output with target output.

$$Error = Target - Output$$

Step 4: Backward Pass

- Error is propagated from output layer to hidden layer.
- Contribution of each neuron to the error is determined.

Step 5: Update Weights

- Weights are adjusted to reduce error.

$$New\ Weight = Old\ Weight + \eta \times Error \times Input$$

where:

- η = Learning Rate

Step 6: Repeat

- Continue until error becomes very small or desired accuracy is achieved.

**Given**

- Input $x = 2$
- Initial Weight $w = 0.5$
- Learning Rate $\eta = 0.1$
- Target Output = 1

Step 1: Initialize Weight

Given:

$$w = 0.5$$

Step 2: Feed Forward

Calculate output:

$$\text{Output} = \text{Input} \times \text{Weight}$$

$$\text{Output} = 2 \times 0.5$$

$$\text{Output} = 1$$

Step 3: Calculate Error

$$\text{Error} = \text{Target} - \text{Output}$$

$$\text{Error} = 1 - 1$$

$$\text{Error} = 0$$

Since the prediction is already correct, no weight update is required.



Input

3

-2

Output

1

0

- **Use**

Early perceptrons

Simple yes/no decisions

Limitation

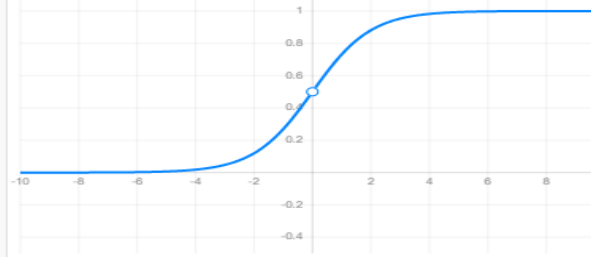
Cannot support gradient-based learning.



3. Sigmoid Function

Formula

$$f(x) = \frac{1}{1+e^{-x}}$$



Output Range

0 to 1

Example

Input	Output
0	0.5
2	0.88
-2	0.12



Advantages

- Smooth curve.
- Useful for probability prediction.
- Widely used in backpropagation examples.

Limitation

- Suffers from vanishing gradient problem.



4. Tanh Function

Formula

$$f(x) = \tanh(x)$$

Output Range

−1 to 1

Example

Input	Output
0	0
Large Positive	≈ 1
Large Negative	≈ -1

Advantages

- Outputs centered around zero.
- Often performs better than sigmoid.



5. ReLU (Rectified Linear Unit)

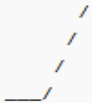
Formula

$$f(x) = \max(0, x)$$

Output

Input	Output
5	5
-3	0

Graph





- **Advantages**

Simple and fast.

Most popular activation function in deep learning.

Reduces vanishing gradient problem.



Step 1: Initialize Weight

$$w = 0.4$$

Step 2: Feed Forward

$$Output = Input \times Weight$$

$$Output = 2 \times 0.4$$

$$Output = 0.8$$

Step 3: Calculate Error

$$Error = Target - Output$$

$$Error = 1 - 0.8$$

$$Error = 0.2$$



Step 4: Backward Pass

The network realizes it is short by 0.2.

Error is sent backward to adjust the weight.

$$Error = 0.2$$

Step 5: Update Weight

$$\begin{aligned} New\ Weight &= Old\ Weight + \eta \times Error \times Input \\ &= 0.4 + (0.1 \times 0.2 \times 2) \\ &= 0.4 + 0.04 \\ &= 0.44 \end{aligned}$$

Step 6: Repeat

Perform another forward pass using the updated weight.

$$Output = 2 \times 0.44$$

$$Output = 0.88$$

The output has moved closer to the target value of 1.



Given

Input $x = 2$

Weight $w = 0.4$

Learning Rate $\eta = 0.1$

Target Output $t = 1$

Sigmoid Activation Function

$$f(x) = \frac{1}{1 + e^{-x}}$$



Step 1: Initialize Weight

$$w = 0.4$$

Step 2: Feed Forward

Calculate Net Input

$$net = x \times w$$

$$net = 2 \times 0.4$$

$$net = 0.8$$

Apply Sigmoid Function

$$Output = \frac{1}{1 + e^{-0.8}}$$

$$Output \approx 0.690$$

Step 3: Calculate Error

$$Error = Target - Output$$

$$Error = 1 - 0.690$$

$$Error = 0.310$$



Step 4: Backward Pass

For sigmoid neurons, we calculate:

$$\delta = Error \times Output \times (1 - Output)$$

Substituting values:

$$\begin{aligned}\delta &= 0.310 \times 0.690 \times (1 - 0.690) \\ &= 0.310 \times 0.690 \times 0.310 \\ \delta &\approx 0.066\end{aligned}$$

Step 5: Update Weight

Use the backpropagation weight update rule:

$$w_{new} = w + \eta \delta x$$

Substitute values:

$$\begin{aligned}w_{new} &= 0.4 + (0.1 \times 0.066 \times 2) \\ &= 0.4 + 0.0132 \\ &= 0.4132\end{aligned}$$

Step 6: Repeat (Forward Pass Again)

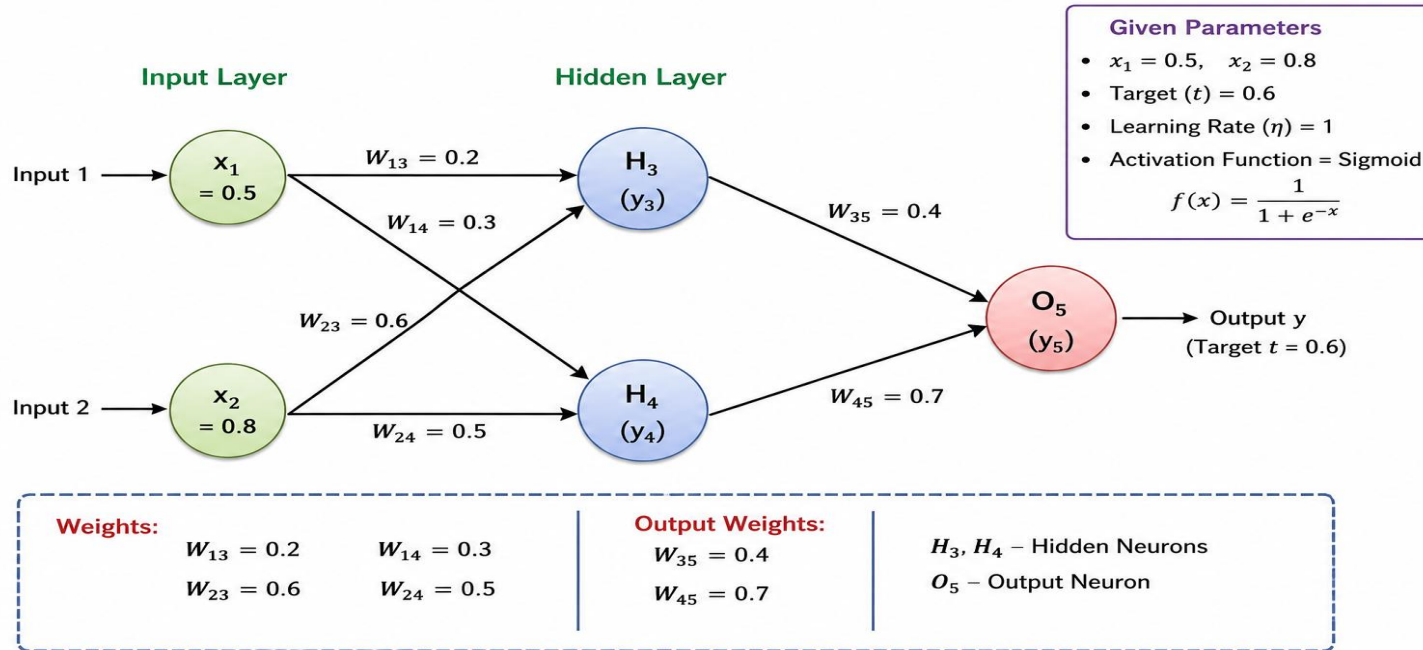
Using the updated weight:

$$\begin{aligned}net &= 2 \times 0.4132 \\ &= 0.8264\end{aligned}$$

Apply sigmoid:

$$\begin{aligned}Output &= \frac{1}{1 + e^{-0.8264}} \\ Output &\approx 0.696\end{aligned}$$

Neural Network for Backpropagation Example





Given

Inputs: $x_1 = 0.5$, $x_2 = 0.8$

Target Output $t = 0.6$

Learning Rate $\eta = 1$

Weights: $W_{13}=0.2$, $W_{23}=0.6$, $W_{14}=0.3$, $W_{24}=0.5$, $W_{35}=0.4$, $W_{45}=0.7$

Activation Function: Sigmoid $f(x)=1/(1+e^{-x})$

Step 1: Forward Pass - Hidden Neuron H3

$$\text{net}_3 = (0.5 \times 0.2) + (0.8 \times 0.6) = 0.58$$

$$y_3 = 1/(1+e^{-0.58}) = 0.6411$$

Step 2: Forward Pass - Hidden Neuron H4

$$\text{net}_4 = (0.5 \times 0.3) + (0.8 \times 0.5) = 0.55$$

$$y_4 = 1/(1+e^{-0.55}) = 0.6341$$

Step 3: Output Neuron O5

$$\text{net}_5 = (0.6411 \times 0.4) + (0.6341 \times 0.7) = 0.7003$$

$$y_5 = 1/(1+e^{-0.7003}) = 0.6683$$



Step 4: Error Calculation

$$\text{Error} = t - y_5$$

$$\text{Error} = 0.6 - 0.6683 = -0.0683$$

Step 5: Output Delta

$$\delta_5 = (t - y_5)y_5(1 - y_5)$$

$$\delta_5 = (-0.0683)(0.6683)(0.3317)$$

$$\delta_5 = -0.0151$$

Step 6: Update Output Weights

$$W_{35}(\text{new}) = 0.4 + (-0.0151)(0.6411) = 0.3903$$

$$W_{45}(\text{new}) = 0.7 + (-0.0151)(0.6341) = 0.6904$$

Step 7: Hidden Layer Deltas

$$\delta_3 = (0.6411)(0.3589)(-0.0151)(0.4) = -0.00139$$

$$\delta_4 = (0.6341)(0.3659)(-0.0151)(0.7) = -0.00245$$



Step 8: Update Hidden Weights

$$W_{13}(\text{new}) = 0.2 + (-0.00139)(0.5) = 0.1993$$

$$W_{23}(\text{new}) = 0.6 + (-0.00139)(0.8) = 0.5989$$

$$W_{14}(\text{new}) = 0.3 + (-0.00245)(0.5) = 0.2988$$

$$W_{24}(\text{new}) = 0.5 + (-0.00245)(0.8) = 0.4980$$

Final Results

$$y_3 = 0.6411$$

$$y_4 = 0.6341$$

$$y_5 = 0.6683$$

$$\text{Error} = -0.0683$$